

PIXIU TECH LLP

Robotics Lab Book

Class 8 | Measurement, Automation and Mobile Robotics

Measure a person's height without touching them, build a bin that opens by itself, and finish the year with a robot that drives on its own.

FIVE UNITS	Introduction Arduino IDE Basic Intermediate Final
FIFTEEN SESSIONS	One session = one class
THREE PROJECTS	Height Station - Smart Dustbin - Obstacle-Avoiding Robot
BOARD	Arduino Uno
THIS BOOK BELONGS TO	
SCHOOL / SECTION	

Contents

Fifteen sessions. One session is one class period.

Unit 1 - Introduction	3
Session 1 - Machines That Measure, and What Is in Your Box	3
Session 2 - Circuits, Breadboards and Signals	5
Check Yourself - Unit 1	9
Unit 2 - The Arduino IDE	9
Session 3 - The Board, the Software and the First Upload	9
Session 4 - Numbers, Timing and the Serial Monitor	11
Session 5 - Loops, Functions and Motors	12
Check Yourself - Unit 2	14
Unit 3 - Basic Project: Height Measurement Station	14
Session 6 - How an Ultrasonic Sensor Really Works	14
Session 7 - From Distance to Height	16
Session 8 - Measuring the Class and Recording Error	18
Check Yourself - Unit 3	20
Unit 4 - Intermediate Project: Smart Contactless Dustbin	20
Session 9 - Servo Motors and the Lid Mechanism	20
Session 10 - Building and Coding the Dustbin	22
Session 11 - False Triggers and How to Beat Them	24
Check Yourself - Unit 4	26
Unit 5 - Final Project: Obstacle-Avoiding Robot	26
Session 12 - Motors, the H-Bridge and Battery Safety	26
Session 13 - Building the Drive Base	28
Session 14 - Adding the Eyes and the Algorithm	30
Session 15 - Arena Testing and Your Demonstration	32
Check Yourself - Unit 5	34
Extra Challenges and Twenty Revision Questions	34
Words to Know	36
Answer Key	37
My Project Log	39

When you see	It means
Today you will	The three things this class is about. Read it before you start.
Try This	A short activity to do right now, at your desk, with your kit.
Watch Out	A mistake almost everyone makes. Read it before you wire.
Did You Know	A fact worth taking home and telling someone.
Dark box	Arduino code. Type it exactly - every bracket and semicolon matters.
Challenge	For when you finish early and want something harder.

UNIT 1

Introduction

Sessions 1 and 2

Every machine in this book has to answer one question well: *how far?* Measuring accurately turns out to be harder, and far more interesting, than it sounds. This unit gives you the electrical foundation you will need, and then everything after it is building.

By the end of this unit you will be able to:

- Explain the Sense - Think - Act idea with your own example.
- Name every component in your kit and say what it does.
- Build a circuit on a breadboard and explain why it works.
- Tell a digital signal from an analog one, and say which pins handle each.

SESSION 1

one class

Machines That Measure, and What Is in Your Box

Today you will

- Understand why measurement is the heart of robotics.
- Learn the names of all twelve components in your kit.
- Agree the rules of the robotics lab.

A robot is any machine that can **sense** something about the world, **decide** what that means, and then **act** on the decision. The automatic door at a shop, the washing machine that slows down when the drum is unbalanced, and the reversing camera in a car are all robots by that definition, even though none of them looks like one.

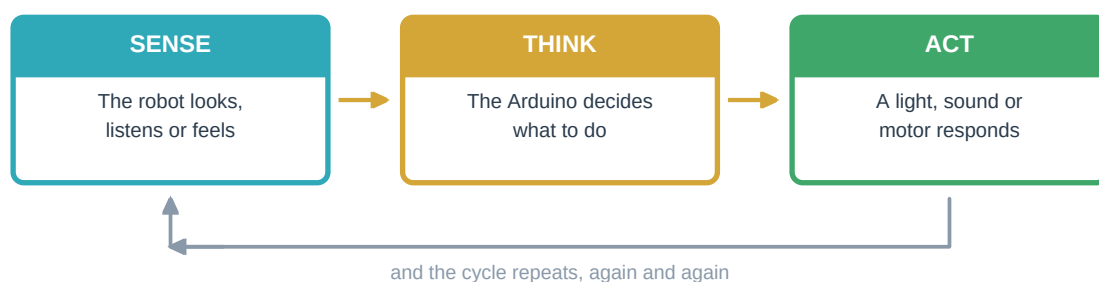


Figure 1.1 - Every robot in the world follows this same three-step loop

This year the **sense** step gets serious. Deciding whether a room is dark is easy - you only need to be roughly right. But if your robot thinks a wall is 40 centimetres away when it is really 20, it will drive straight into it. Once a machine has to move, accuracy stops being a nice extra and becomes the whole job.

Try This

Guess the width of your desk in centimetres. Write your guess down. Now measure it with a ruler and write the real value beside it. The difference between the two numbers has a name that you will use all year: it is called **error**. Engineers do not try to have zero error. They try to know how big their error is.

1.1 Open the Box

Lay everything on the desk before you plug anything in. Learn the names first - it is impossible to ask for help with 'that small blue thing'. Four of these will be completely new to most of you: the ultrasonic sensor, the servo motor, the motor driver and the chassis.

ARDUINO KIT COMPONENTS – CLASS 7

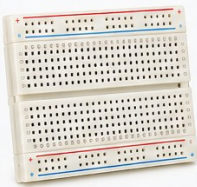
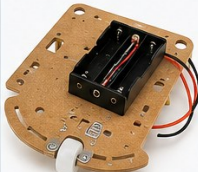
1 Arduino Uno R3 board  Unit 2 the brain	2 400-point breadboard  Unit 1 wiring base	3 5mm LEDs (blue, white, orange)  Unit 1 range indicator	4 220 ohm resistors  Unit 1 protect the LED	5 Jumper wires (M-M, M-F)  Unit 1 the roads	6 Arduino USB A-B cable  Unit 2 power + upload
7 HC-SR04 ultrasonic sensor  Unit 3 measures distance	8 SG90 servo motor  Unit 4 lifts the lid	9 Buzzer module (3-5V)  Unit 4 sound output	10 L298N motor driver module  Unit 5 controls the motors	11 100 RPM BO motor with yellow wheel  Unit 5 drives the robot	12 Robot chassis with caster wheel and battery holder  Unit 5 the robot body

Figure 1.1 - Your kit. Learn these twelve names before the next class.

Part	What it does	First used in
Arduino Uno board	The brain. It stores your program and runs it forever, even after the power has been off.	Session 3
USB A-B cable	Carries your program into the board and powers it while you test.	Session 3
400-point breadboard	Joins components together without any soldering.	Session 2
Jumper wires	The roads between parts. Male-to-male and male-to-female.	Session 2
5mm LEDs	Small lamps. Here they become a distance indicator you can read at a glance.	Session 2
220 ohm resistors	Limit the current so an LED is not destroyed.	Session 2
HC-SR04 ultrasonic sensor	Measures distance by timing an echo. The star of this book.	Session 6
SG90 servo motor	Turns to an exact angle and holds it there.	Session 9
Buzzer module	Makes sound, so the machine can warn people.	Session 10
L298N motor driver	A power amplifier for motors. The Arduino cannot drive them directly.	Session 12
100 RPM BO motor with wheel	Turns the wheels. You need two of these.	Session 13
Chassis, caster wheel, battery holder	The robot's body and its power supply.	Session 13

1.2 Rules of the Robotics Lab

These eight rules are not decoration. Each one exists because somebody, somewhere, damaged equipment by ignoring it. Two of them are new this year, because this year you will work with batteries and motors.

- 1 Always unplug the USB cable before you change any wire.
- 2 Check the long leg of every LED and the + side of every module before power goes on.
- 3 Never run a wire straight from 5V to GND. That is a short circuit and it can kill the board.
- 4 **Never connect a motor directly to an Arduino pin.** It draws far more current than the pin can supply and will damage the board permanently.
- 5 **Disconnect the battery pack before rewiring anything**, and never leave it connected at the end of a session.
- 6 Lift a robot off the floor before uploading new code, so it cannot drive away mid-upload.
- 7 If anything smells hot or feels hot, disconnect it at once and call your trainer.
- 8 Count every component back into the kit box before you leave.

Watch Out

Rule 4 is the one that ends most school robotics kits. An Arduino pin can supply about 20 milliamps safely. A BO motor wants several hundred. That gap is exactly why the L298N motor driver exists, and you will meet it properly in Unit 5.

New word	What it means
Robot	A machine that senses, decides and acts.
Error	The difference between a measured value and the true value.
Current	How much electricity flows each second. Measured in amps.

SESSION 2

one class

Circuits, Breadboards and Signals

Today you will

- Refresh how a circuit works and why it must be a loop.
- Use a breadboard correctly and light an LED with no code.
- Learn the difference between digital and analog signals.

Electricity behaves much like water in a pipe. The battery is the pump, the wires are the pipes, and the LED is a small fountain at the end. **Voltage** is how hard the pump pushes, measured in volts. **Current** is how much flows each second, measured in amps. **Resistance** is anything that makes flowing harder, measured in ohms.

The rule that matters most: **electricity only flows in a complete loop**. It must leave the supply, pass through every component, and return. Break the loop anywhere and everything stops. A switch is simply a gap you can open and close deliberately.

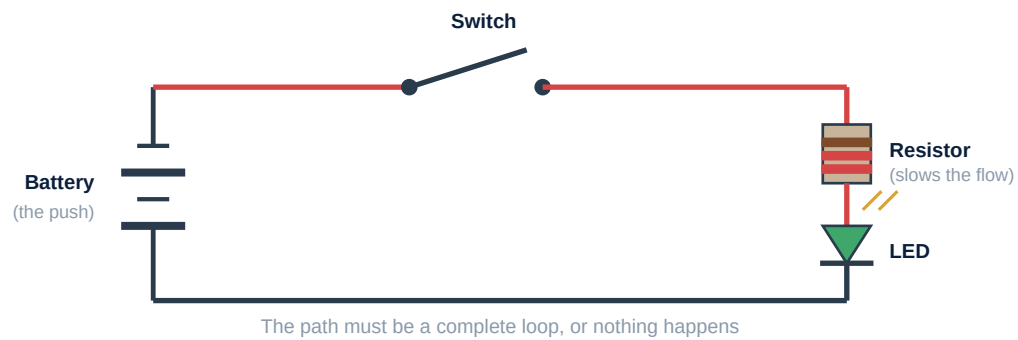
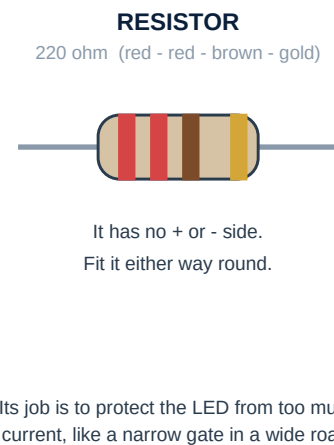
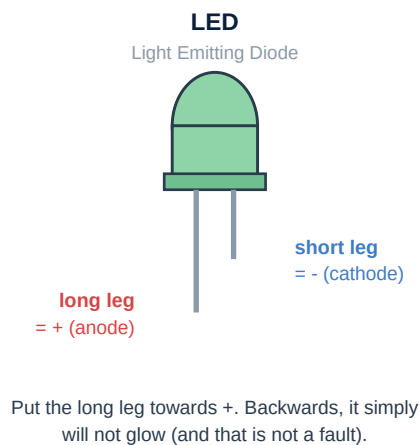


Figure 1.2 - A simple circuit: battery, switch, resistor and LED

**Watch Out**

An LED has a long leg and a short leg. Long leg towards +, short leg towards -. If an LED does not glow, turn it around before checking anything else. And it always needs a resistor beside it: connect one straight to 5V and it will glow brilliantly for about a second, then never again.

2.1 Inside the Breadboard

A breadboard looks like a plain block of holes but is full of hidden metal strips. Two holes joined by a strip behave exactly as though you had twisted those wires together. Knowing which holes are secretly joined is the commonest reason a beginner's circuit does nothing at all.

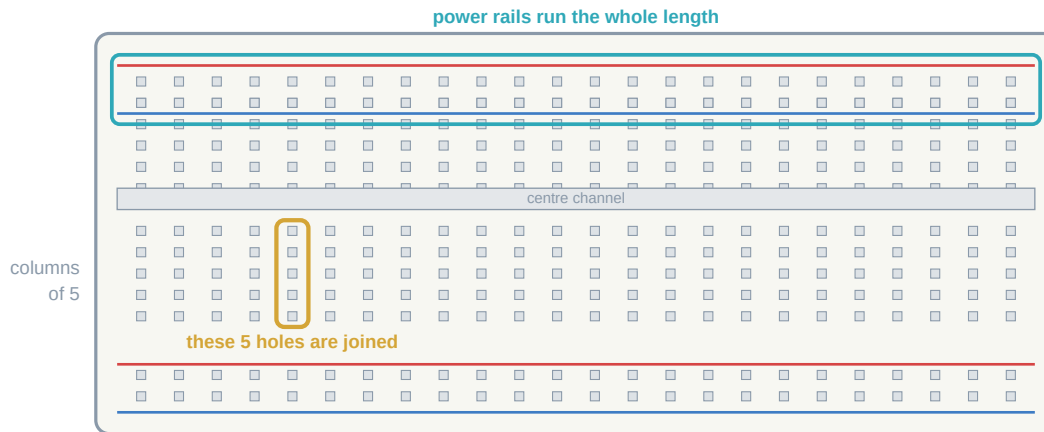


Figure 1.3 - Inside a breadboard: which holes are secretly connected

- The **long side rails** marked + and - run the whole length. Use them for power and ground.
- In the middle, holes are joined in **columns of five**, up and down only, never across.
- The **centre channel** splits the board in two. Nothing crosses it.
- So two legs in the same column of five are connected; two legs in different columns are not.

Try This

Push an LED into the breadboard with its legs in different columns. Put a 220 ohm resistor in the same column as the long leg. Wire the resistor's free end to 5V and the short leg's column to GND. Plug in the USB - it glows, with no code at all. Now pull out the GND wire and watch it die. You have just proved the loop rule with your own hands.

2.2 Two Kinds of Signal

A light switch has exactly two positions, on and off, with nothing in between. Engineers call that a **digital** signal. A fan regulator moves smoothly through many positions - that is an **analog** signal. The real world is mostly analog: temperature climbs, light fades, distance changes continuously.

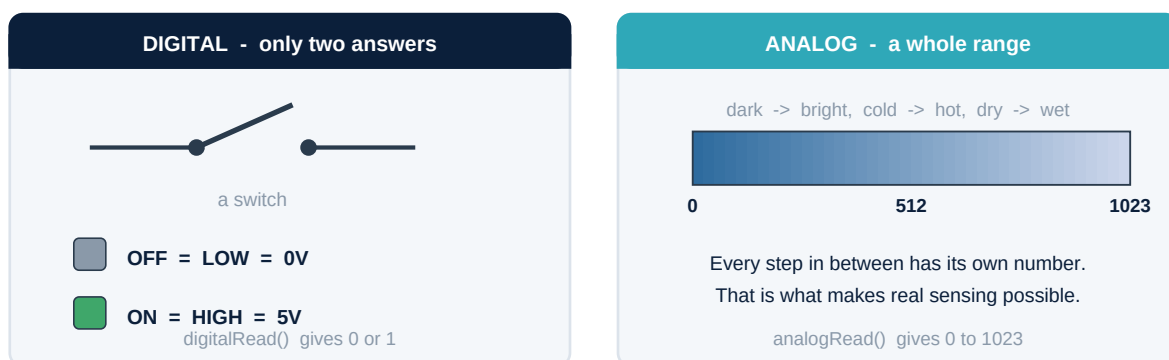


Figure 1.2 - The big idea of Class 7: numbers instead of yes-or-no

The Arduino handles both. Digital pins D0 to D13 deal in HIGH and LOW. Analog pins A0 to A5 measure a voltage and report a number from 0 to 1023. Here is the interesting part: the HC-SR04 you will use all year is a **digital** sensor, even though distance is an analog quantity. It does its own measuring inside and hands you the answer as the *length of a pulse*. Working out what that pulse means is your job in Unit 3.

New word	What it means
Voltage	How hard electricity is pushed. Measured in volts.
Resistance	Anything that makes current flow harder. Measured in ohms.
Ground (GND)	The return path. Every circuit needs one.
Digital / Analog	Only two values / any value within a range.

CHECK WHAT YOU LEARN'T

Check Yourself - Unit 1**A. Choose the correct answer**

- Electricity flows only when the circuit is
(a) broken (b) hot (c) a complete loop (d) made of plastic
- The long leg of an LED connects towards
(a) GND (b) the minus side (c) the plus side (d) either side
- In the middle of a breadboard, holes are joined in
(a) columns of five (b) rows of ten (c) no pattern (d) pairs
- A signal that can be any value in a range is called
(a) digital (b) analog (c) electric (d) binary
- An Arduino pin can safely supply about
(a) 20 milliamps (b) 2 amps (c) 20 amps (d) no current
- The difference between a measured value and the true value is called
(a) current (b) error (c) voltage (d) resistance

B. Fill in the blanks

- The push that moves electricity along is called _____.
- Analog pins on the Arduino Uno are labelled _____ to _____.
- A motor must never be connected directly to an _____ pin.
- The battery pack must be _____ before any rewiring.

C. Answer in one or two lines

- Name one machine at home that follows Sense - Think - Act, and explain each step.
- Why does accuracy matter far more once a machine starts to move?
- Your LED does not glow. Write the first three things you would check, in order.
- Explain why a resistor is needed beside an LED.

UNIT 2

The Arduino IDE

Sessions 3, 4 and 5

Three skills separate this year's work from simple blinking lights: timing something in microseconds, doing arithmetic on the result, and wrapping the whole job inside an instruction you wrote yourself. This unit teaches all three.

By the end of this unit you will be able to:

- Upload a program and use the Serial Monitor to watch live values.
- Use `pulseIn()` to measure how long a pulse lasts.
- Write your own function and call it from `loop()`.
- Explain why a motor needs a driver and what an H-bridge does.

SESSION 3

one class

The Board, the Software and the First Upload

Today you will

- Learn the parts of the Arduino Uno board.
- Set the board and port and upload Blink.
- Prove your whole setup works before any project begins.

The Arduino Uno is a small computer with no screen and no keyboard. It runs one program at a time - the one you give it - keeps it in memory when the power is removed, and starts running again the instant power returns. That is why your finished robot still works when you unplug it from the laptop and run it from a battery.

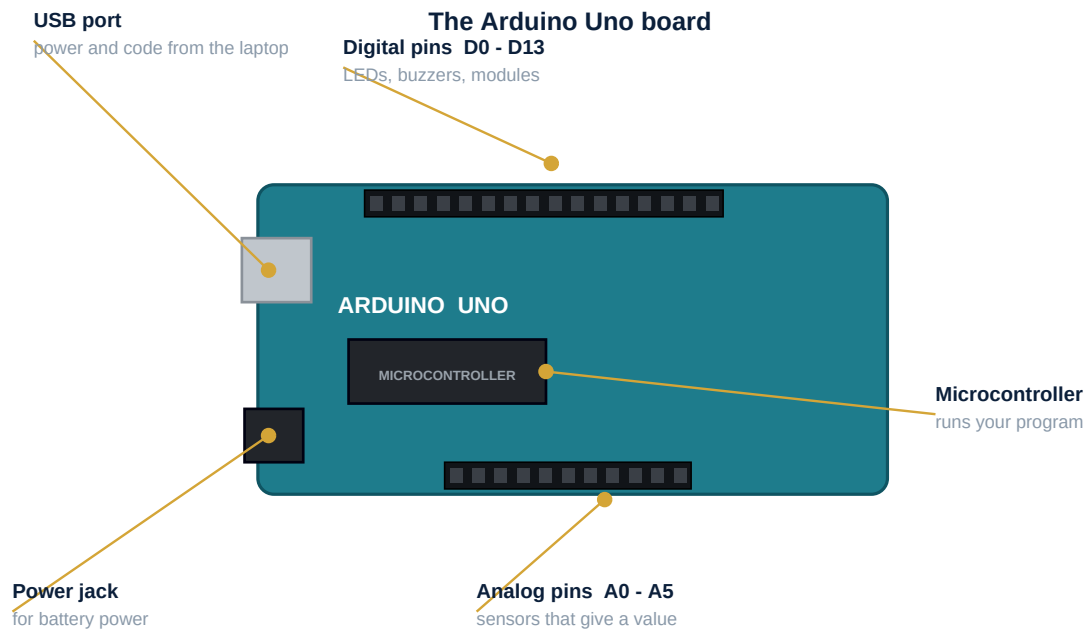


Figure 2.1 - Know your board before you wire anything to it

Group of pins	What is special about them	Used in this book for
D0 to D13	Plain on-or-off inputs and outputs	the ultrasonic sensor, LEDs, the buzzer
~3, ~5, ~6, ~9, ~10, ~11	Can produce a varying output, not just on or off	the servo, and motor speed control
A0 to A5	Measure a voltage and report 0 to 1023	extension work only, this year
5V and GND	Power out to your modules	everything, always

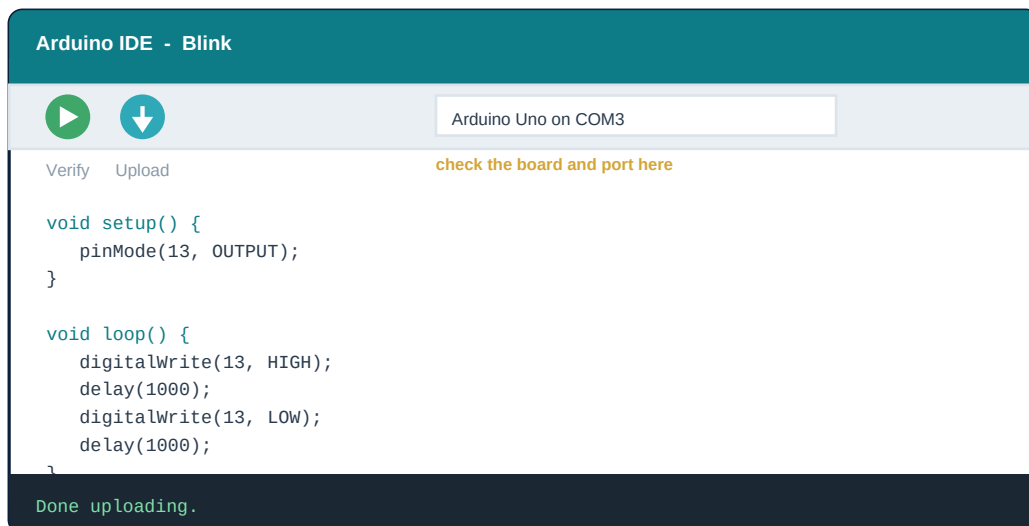


Figure 2.2 - Verify checks your code; Upload sends it to the board

Watch Out

Before your first upload, check **Tools > Board** says Arduino Uno and **Tools > Port** has a port selected. When an upload fails, this is the first thing to check, every single time.

Blink.ino - the test that proves everything works

```
int led = 13;

void setup() {
  pinMode(led, OUTPUT);    // this pin will send power out
}

void loop() {
  digitalWrite(led, HIGH); // switch it on
  delay(1000);             // wait one second
  digitalWrite(led, LOW);  // switch it off
  delay(1000);
}
```

SESSION 4

one class

Numbers, Timing and the Serial Monitor**Today you will**

- Learn the variable types you will need this year.
- Meet microseconds and the pulseIn() instruction.
- Print live values to the Serial Monitor.

A **variable** is a named box that holds a number. Which type of box you choose matters more this year than it ever has, because the numbers coming back from an ultrasonic sensor can be surprisingly large.

Type	Holds	Biggest value it can store	Use it for
int	A whole number	about 32,767	distances in centimetres
long	A bigger whole number	over 2,000,000,000	times in microseconds
float	A number with a decimal point	very large	heights, averages

Type	Holds	Biggest value it can store	Use it for
boolean	Only true or false	-	flags such as objectSeen

Watch Out

An echo from an object two metres away takes roughly 11,600 microseconds to return - that fits in an int. But if the sensor sees nothing at all it waits far longer, and the number overflows an int and turns negative. That is why the timing value in this book is always stored in a **long**.

4.1 Time, in Slices of a Millionth

You already know `delay(1000)` waits one second, because it counts in **milliseconds** - thousandths of a second. Sound travels far too fast for that to be useful here. Over one millisecond, sound covers about 34 centimetres, so a millisecond-accurate clock could not tell 10 cm from 40 cm.

So the Arduino offers a finer ruler: the **microsecond**, one millionth of a second. Sound travels about 0.034 centimetres in that time - roughly a third of a millimetre. Now you can measure properly. Two instructions matter: `delayMicroseconds()` waits a very short time, and `pulseIn(pin, HIGH)` waits for a pin to go HIGH and reports *how many microseconds it stayed HIGH*.

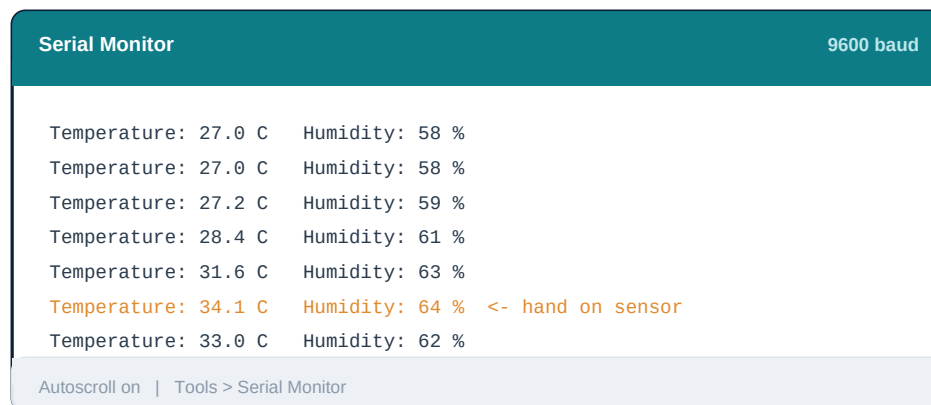


Figure 2.3 - The Serial Monitor is your window into the machine's mind

Try This

Add `Serial.begin(9600);` to `setup()` and `Serial.println(millis());` to `loop()` with a `delay(500);` after it. Open the Serial Monitor and watch. `millis()` reports how long the board has been running. Press the reset button and watch the count start again from zero.

SESSION 5

one class

Loops, Functions and Motors

Today you will

- Use a for loop to repeat something a set number of times.
- Write your own function and call it.
- Understand why a motor needs a driver, not a pin.

A **for loop** repeats a block a fixed number of times. It has three parts inside its brackets: where to start, how long to keep going, and what to change each time round.

The shape of a for loop

```
for (int i = 0; i < 5; i++) { // start at 0, stop below 5, add 1 each time
```

```
Serial.println(i);          // prints 0, 1, 2, 3, 4
}
```

Far more important this year is the **function** - an instruction you write yourself. Taking five lines of measuring code and giving them one clear name turns an unreadable loop into something a stranger could follow. Professional programmers do this constantly, and from Unit 3 onwards so will you.

The shape of a function

```
int readDistance() {          // int means: this hands back a whole number
  // ... the measuring steps go in here ...
  return answer;              // return means: hand this value back
}

void loop() {
  int d = readDistance();     // one clear line instead of five confusing ones
}
```

5.1 Why a Motor Cannot Touch a Pin

An Arduino output pin is designed to signal, not to supply power. It can deliver about 20 milliamps safely. A BO motor starting under load can demand several hundred milliamps - more than ten times as much. Connect one directly and the pin does not simply fail; it burns out, and the damage is permanent.

The solution is a **motor driver**. Think of it as a relay race. The Arduino sends a tiny signal saying *turn this motor forwards*. The driver, connected to a battery, does the heavy lifting. The Arduino never touches the motor current at all - and that is exactly the point.

Did You Know

The same idea scales all the way up. A driver in an electric car takes a signal from a small computer and switches hundreds of amps to the wheels. Your L298N and that system are solving exactly the same problem, just at different sizes.

New word	What it means
Microsecond	One millionth of a second.
pulseIn()	Measures how long a pin stays HIGH, in microseconds.
Function	An instruction you write yourself and then reuse.
return	Hands a value back from a function to whoever called it.
Motor driver	A circuit that switches heavy current on a small signal's command.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 2**A. Choose the correct answer**

- One microsecond is
(a) a thousandth of a second (b) a millionth of a second (c) a minute (d) a second
- `pulseIn(echo, HIGH)` measures
(a) voltage (b) how long the pin stays HIGH (c) distance directly (d) temperature
- A timing value in microseconds should be stored in a
(a) boolean (b) int (c) long (d) char
- A function that hands a value back must use
(a) delay (b) return (c) print (d) loop
- A motor must not be connected directly to an Arduino pin because
(a) it is too slow (b) it draws far too much current (c) it is too heavy (d) the pin is analog
- `for (int i = 0; i < 5; i++)` runs the block
(a) 4 times (b) 5 times (c) 6 times (d) forever

B. Fill in the blanks

- Sound travels about _____ centimetres in one microsecond.
- A _____ is an instruction you write yourself and reuse.
- The instruction that waits a very short time is `delay_____()`.
- A motor gets its power from the _____, not from the Arduino.

C. Answer in one or two lines

- Explain why milliseconds are too coarse for measuring an echo.
- Write the first line of a function called `readDistance()` that returns a whole number.
- Explain the motor driver using the relay race idea, in your own words.
- Why might an int variable turn negative when the sensor sees nothing?

UNIT 3

Basic Project: Height Measurement Station

Sessions 6, 7 and 8

Your first machine measures a person's height without a tape and without touching them. It sounds like magic. It is actually one subtraction, wrapped around a sensor that is really a very fast stopwatch.

By the end of this unit you will be able to:

- Explain time-of-flight measurement in your own words.
- Apply a formula inside a program to get a derived quantity.
- Write and call your own `readDistance()` function.
- Measure your machine's error honestly and reduce it by averaging.

SESSION 6

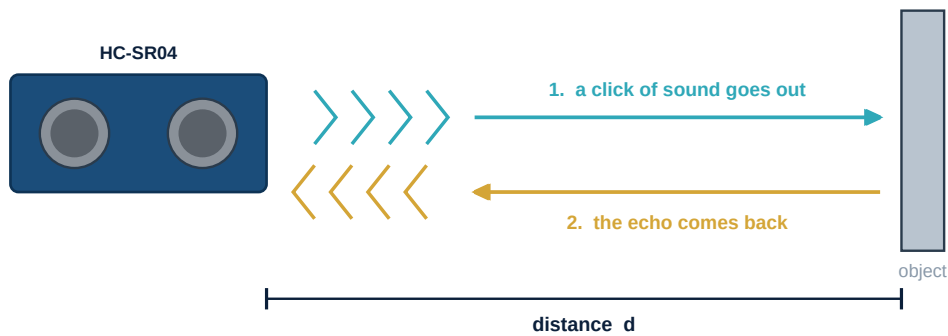
one class

How an Ultrasonic Sensor Really Works

Today you will

- Understand echo and time of flight.
- Learn the four-step trigger sequence.
- Get your first raw readings onto the Serial Monitor.

The HC-SR04 has two metal cylinders. One is a tiny speaker, the other a tiny microphone. The speaker emits a burst of sound at about 40,000 vibrations per second - far above the roughly 20,000 that human ears can detect, which is why the sensor is called **ultrasonic**. The microphone then listens for that burst to come back.

How an ultrasonic sensor measures without touching

$$\text{distance} = \text{time} \times 0.034 / 2$$

time is in microseconds | 0.034 cm is how far sound travels in one microsecond
we divide by 2 because the sound made the journey twice - there and back

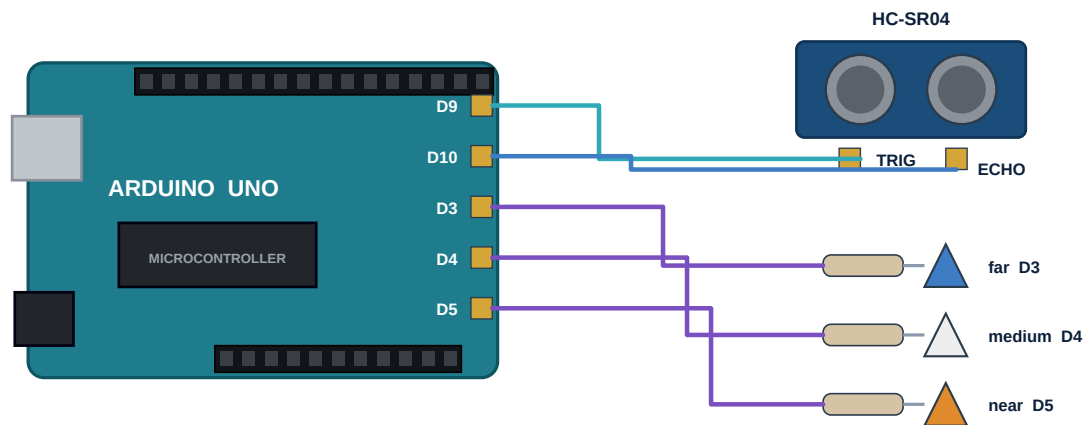
Figure 3.1 - Time of flight: the sensor is really a very fast stopwatch

Work through the arithmetic once by hand and it will never confuse you again. Suppose the echo takes **1160 microseconds** to return. Sound covers 0.034 cm per microsecond, so it travelled $1160 \times 0.034 = 39.4$ cm in total. But that journey was out *and* back, so the object is half that away: **19.7 cm**.

To take a reading you must follow a strict four-step sequence. The 10-microsecond pulse is the sensor's instruction to fire; anything shorter may be ignored.

- 1 Set TRIG LOW for 2 microseconds, so the line starts clean.
- 2 Set TRIG HIGH for exactly 10 microseconds. This is the command to fire.
- 3 Set TRIG LOW again.
- 4 Call `pulseIn(ECHO, HIGH)`, which waits and reports how long the echo pin stayed HIGH.

Distance meter - connections



The sensor also needs VCC to 5V and GND to GND.

Figure 3.2 - Two wires for the sensor, three for the indicator lamps

Sensor pin	Arduino pin	Why
VCC	5V	Power for the sensor
TRIG	D9	Your 10 microsecond command to fire
ECHO	D10	The pin whose HIGH time you measure
GND	GND	Shared return path
Far / medium / near LEDs	D3 / D4 / D5 through 220 ohm	A bar you can read at a glance

Watch Out

TRIG and ECHO are easy to swap, and the symptom is unmistakable: every reading comes back as 0. If that happens, swap the two wires before you touch the code.

SESSION 7

one class

From Distance to Height

Today you will

- Mount the sensor at a known height, facing down.
- Turn a distance reading into a height with one subtraction.
- Write and use your own readDistance() function.

Here is the trick that turns a distance meter into a height meter. Fix the sensor at the top of a stand, pointing straight down, and measure that mounting height once with a tape - call it **H**. When somebody stands underneath, the sensor reports the gap between itself and the top of their head - call that **d**. Their height is simply **H minus d**.

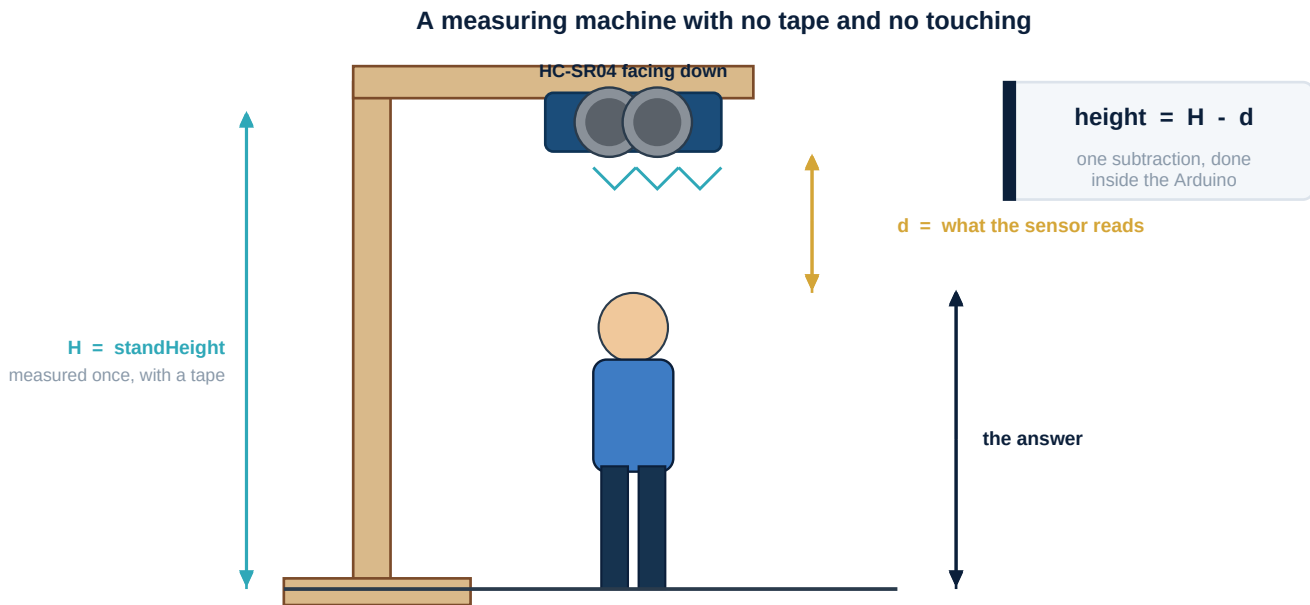


Figure 3.3 - Fix the sensor at a known height and let the maths do the rest

This is a genuinely important idea in engineering, and it has a name: a **derived quantity**. The sensor never measures height at all. It measures a gap, and your program turns that into something a human actually wants to know. Most of the useful numbers in the world are derived this way.

HeightStation.ino

```
int trig = 9;
int echo = 10;
int farLed = 3, midLed = 4, nearLed = 5;

float standHeight = 180.0; // measure YOUR stand with a tape and change this

void setup() {
  pinMode(trig, OUTPUT);
  pinMode(echo, INPUT);
  pinMode(farLed, OUTPUT);
  pinMode(midLed, OUTPUT);
  pinMode(nearLed, OUTPUT);
  Serial.begin(9600);
}

int readDistance() { // our own instruction
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10); // the command to fire
  digitalWrite(trig, LOW);

  long t = pulseIn(echo, HIGH); // microseconds the echo pin stayed HIGH
  return t * 0.034 / 2; // there and back, so halve it
}

void loop() {
  int d = readDistance();
  float height = standHeight - d;

  Serial.print("gap: ");
  Serial.print(d);
  Serial.print(" cm height: ");
  Serial.print(height);
}
```

```

Serial.println(" cm");

digitalWrite(farLed, height < 130); // a short way to write if-else
digitalWrite(midLed, height >= 130 && height < 160);
digitalWrite(nearLed, height >= 160);

delay(500);
}

```

Look at the three LED lines. `height < 130` is a comparison, and a comparison is already true or false - which is exactly what `digitalWrite` wants. So the lamp lights when the comparison is true, with no if statement needed at all. It is a neat shortcut worth knowing, though a full if-else is clearer when the logic gets complicated.

SESSION 8

one class

Measuring the Class and Recording Error

Today you will

- Measure classmates and compare against a tape.
- Record your error honestly in a table.
- Reduce noise by averaging several readings.

Now find out how good your machine actually is. Measure five classmates with your station, then measure the same five with a measuring tape. Fill in both columns and work out the difference each time. Do not adjust anything until the table is complete.

Name	Your machine says (cm)	Tape says (cm)	Error (cm)
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____
Largest error			_____

You will almost certainly find the readings jump about by a centimetre or two even when nobody moves. That wobble is called **noise**, and every real sensor has it. The professional cure is beautifully simple: take several readings quickly and use their average. One odd reading then counts for only a fifth of the answer instead of all of it.

Averaging to beat the noise

```

int readAverage() { // five readings, one steady answer
  long total = 0;
  for (int i = 0; i < 5; i++) {
    total = total + readDistance();
    delay(20);
  }
  return total / 5;
}

```

Watch Out

Hair is a poor reflector of sound - it scatters the echo instead of bouncing it straight back. Thick or curly hair can make a person read several centimetres taller than they are. Place a flat book on the head and your error will drop immediately. Knowing *why* a machine is wrong is worth more marks than pretending it is right.

Challenge

Show the height in feet and inches as well as centimetres. One inch is 2.54 cm. Then use the buzzer to beep once when a stable reading has been held for two seconds, so the person knows they can step away.

New word	What it means
Ultrasonic	Sound too high for human ears to hear.
Time of flight	Measuring distance by timing how long a signal takes to return.
Derived quantity	A useful number worked out from something else that was measured.
Noise	Small random variation in a sensor's readings.
Averaging	Taking several readings and using their mean to reduce noise.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 3**A. Choose the correct answer**

1. An ultrasonic sensor measures distance by
(a) seeing light (b) timing an echo (c) touching the object (d) weighing it
2. The distance formula divides by 2 because
(a) it is easier (b) sound travels there and back (c) the sensor is slow (d) of the resistor
3. The TRIG pulse must be held HIGH for about
(a) 2 microseconds (b) 10 microseconds (c) 10 milliseconds (d) 1 second
4. Every reading comes back as 0. The likely fault is
(a) a dead battery (b) TRIG and ECHO swapped (c) a missing LED (d) wrong baud rate
5. If the stand is 180 cm and the sensor reads 42 cm, the person is
(a) 222 cm (b) 138 cm (c) 42 cm (d) 90 cm
6. Averaging five readings is done to reduce
(a) voltage (b) noise (c) current (d) the delay

B. Fill in the blanks

- 1 Sound travels about _____ cm in one microsecond.
- 2 Height equals standHeight _____ the measured gap.
- 3 A number worked out from another measurement is called a _____ quantity.
- 4 Hair scatters the echo, which makes a person read too _____.

C. Answer in one or two lines

- 1 An echo returns in 900 microseconds. Work out the distance, showing your steps.
- 2 Write down your machine's largest error and one honest reason for it.
- 3 Explain why averaging makes a reading steadier.
- 4 Why is the sensor called a digital sensor when distance is an analog quantity?

UNIT 4

Intermediate Project: Smart Contactless Dustbin

Sessions 9, 10 and 11

Until now your machines have only reported. This one *moves*. A servo lifts a real lid when a hand comes near, holds it, and closes it again - and getting that to feel natural is harder than the code suggests.

By the end of this unit you will be able to:

- Explain how a servo differs from an ordinary motor.
- Design a mechanical linkage that matches the servo's travel.
- Combine a sensor and an actuator into one automatic machine.
- Identify false triggers and remove them.

SESSION 9

one class

Servo Motors and the Lid Mechanism

Today you will

- Learn how a servo knows where it is.
- Test the servo's travel before building anything.
- Design the bin and lid on paper.

An ordinary motor spins as long as you give it power, and it has no idea where it is. A **servo** is different. Inside its plastic case there is a small motor, a gearbox, and - most importantly - a sensor watching the output shaft. You tell the servo an angle, and its own internal circuit keeps adjusting until the shaft is at that angle, then holds it there against the weight of whatever is attached.

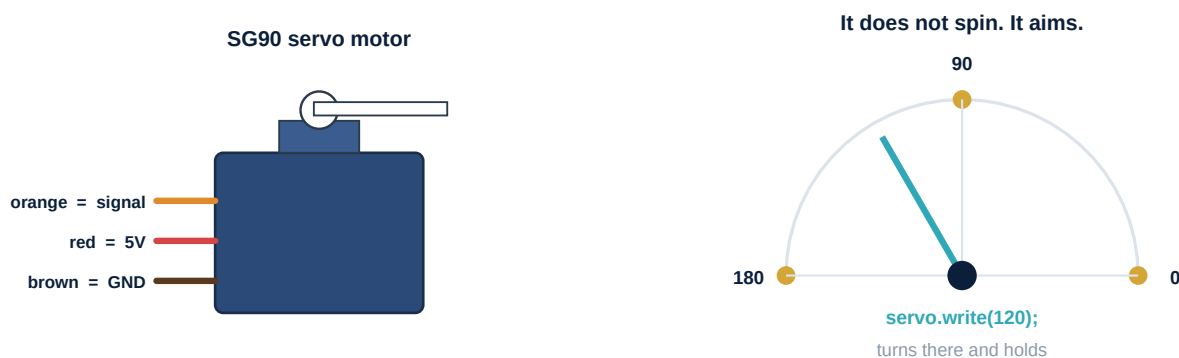


Figure 5.3 - A servo turns to an exact angle between 0 and 180 degrees

Servo wire	Colour on an SG90	Connect to
Signal	Orange or yellow	any digital pin, D6 in this project
Power	Red	5V
Ground	Brown or black	GND, shared with the Arduino

ServoTest.ino - run this before you build anything

```
#include <Servo.h>           // borrow the ready-made servo instructions

Servo lid;

void setup() {
  lid.attach(6);             // the servo's signal wire is on D6
  lid.write(0);              // start closed
}

void loop() {
  lid.write(90);             // open
  delay(2000);
  lid.write(0);             // close
  delay(2000);
}
```

Watch Out

Never force a servo horn round with your fingers. The gears inside are small plastic teeth and they strip easily - and a stripped servo cannot be repaired. Always move it by sending it an angle.

Try This

Before building the bin, run ServoTest and watch the horn carefully. Mark on paper exactly where the arm sits at 0 degrees and at 90 degrees. Your whole lid mechanism has to fit inside that sweep - design around the servo, not the other way round.

SESSION 10

one class

Building and Coding the Dustbin**Today you will**

- Build the bin, hinge the lid and mount the servo.
- Wire the sensor, servo, buzzer and lamp.
- Get the full open-hold-close cycle running.

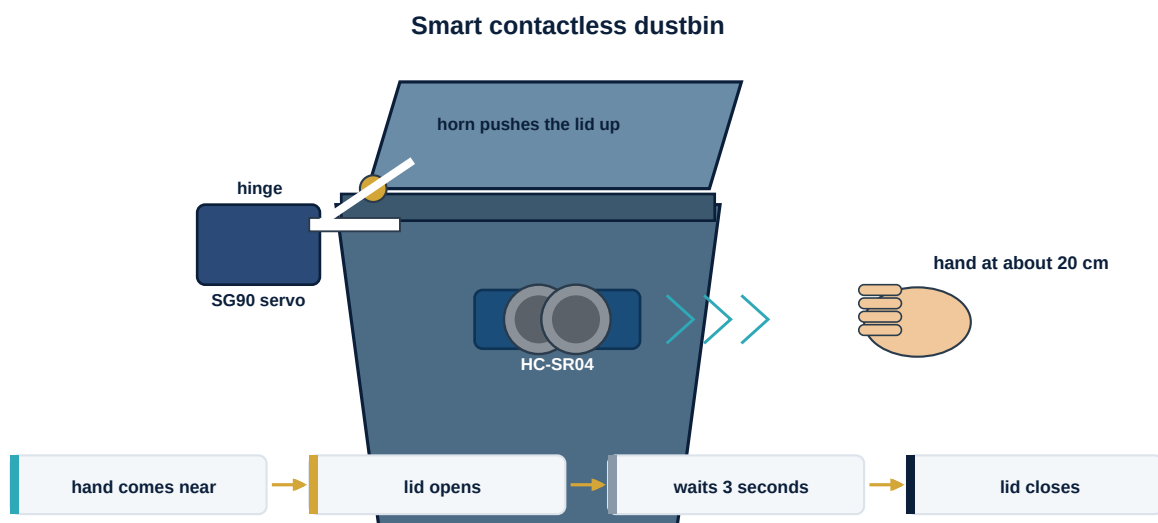


Figure 4.1 - A sensor, a motor, and a lid nobody has to touch

- Keep the lid **light**. A heavy cardboard lid makes the servo strain, buzz and eventually give up.
- Fix the servo body firmly to the bin. If the body moves instead of the horn, nothing works.
- Mount the sensor on the **front face**, pointing outwards and slightly upwards, at about the height a hand approaches.
- Make sure the open lid never blocks the sensor's view - a classic and very confusing fault.

Smart dustbin - connections

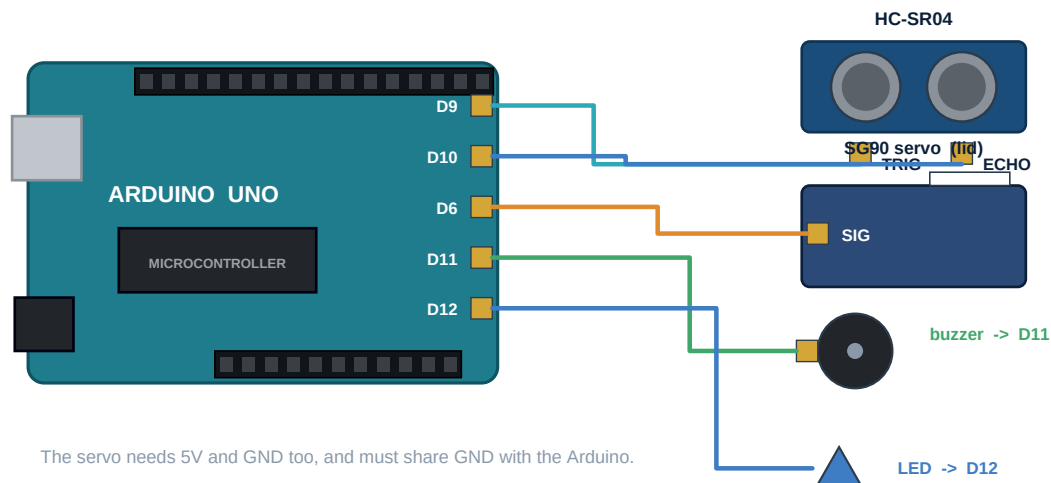


Figure 4.2 - One sensor in, one motor and two alerts out

Component pin	Arduino pin	Why
HC-SR04 TRIG	D9	The command to fire
HC-SR04 ECHO	D10	The pulse you time
Servo signal (orange)	D6	Lid angle
Servo red / brown	5V / GND	Power. GND must be shared with the Arduino
Buzzer +	D11	A short chirp as the lid opens
Blue LED long leg	D12 through 220 ohm	Shows the lid is open

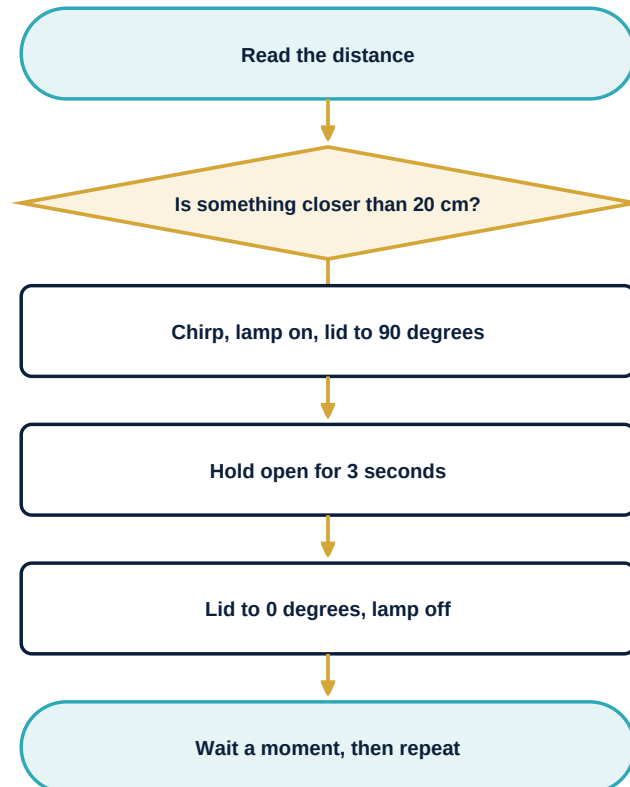


Figure 4.3 - The open-hold-close cycle, drawn before it is coded

SmartDustbin.ino

```

#include <Servo.h>

Servo lid;
int trig = 9, echo = 10, buzzer = 11, lamp = 12;
int openDistance = 20;      // centimetres - tune this yourself

void setup() {
  lid.attach(6);
  pinMode(trig, OUTPUT);
  pinMode(echo, INPUT);
  pinMode(buzzer, OUTPUT);
  pinMode(lamp, OUTPUT);
  lid.write(0);              // start closed
  Serial.begin(9600);
}

int readDistance() {
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long t = pulseIn(echo, HIGH);
  return t * 0.034 / 2;
}

void loop() {
  int d = readDistance();
  Serial.println(d);

  if (d > 0 && d < openDistance) {
    digitalWrite(lamp, HIGH);
    digitalWrite(buzzer, HIGH);
    delay(80);
    digitalWrite(buzzer, LOW);

    lid.write(90);           // open
    delay(3000);             // hold
    lid.write(0);            // close
    digitalWrite(lamp, LOW);
    delay(500);              // settle before looking again
  }

  delay(100);
}

```

Notice `d > 0` in the condition. When the sensor hears no echo at all, `pulseIn` eventually gives up and returns 0, which would otherwise look like *something is touching me* and fling the lid open at random. That one small test removes an entire class of mysterious faults.

SESSION 11

one class

False Triggers and How to Beat Them

Today you will

- Find out what makes the bin open when it should not.
- Test with real users, not just your own hand.
- Tune the threshold and the hold time.

Your bin works. Now find out how it **fails**, because a machine is only finished when you know its weaknesses. A trigger that happens when nobody meant it is called a **false trigger**, and every automatic machine in the world fights them.

What you see	Why it happens	What to do about it
Lid opens with nobody there	The sensor sees the wall, a chair or its own open lid	Aim it outwards and check nothing crosses its view
Lid opens twice for one hand	The hand is still there when the loop looks again	Increase the settle delay after closing
Lid opens very late	Threshold set too close to the bin	Raise openDistance towards 25 or 30
Lid trembles while open	The servo is straining against a heavy lid	Lighten the lid or move the horn closer to the hinge
Random opening at 0 cm	No echo returned and pulseIn gave up	Keep the d > 0 test in your condition

Now test with real users. Ask three classmates who have not seen your bin to throw a paper ball into it, and watch without saying anything. People approach at odd angles, from the side, or too fast. What you learn in those thirty seconds is worth more than an hour of adjusting numbers on your own.

Acceptance test	Result	Tick
Lid opens within 1 second of a hand at 20 cm	_____	■
Lid stays open about 3 seconds	_____	■
Lid closes fully every time, with no trembling	_____	■
Ten approaches in a row, no false trigger	_____	■
Left running for two minutes untouched: stays shut	_____	■

Challenge

Add a second HC-SR04 pointing down inside the bin to measure the fill level. When rubbish rises above a set line, flash the lamp to say the bin needs emptying. Real smart bins in airports do exactly this and send a message to the cleaning staff.

New word	What it means
Servo	A motor that turns to an exact angle and holds it.
Actuator	Any part that moves or acts: a motor, a buzzer, a lamp.
Horn	The plastic arm attached to a servo's shaft.
False trigger	The machine acting when nobody intended it to.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 4**A. Choose the correct answer**

1. A servo differs from an ordinary motor because it
(a) spins faster (b) knows its own angle (c) needs no power (d) is smaller
2. The signal wire of an SG90 servo is usually
(a) red (b) brown (c) orange (d) black
3. `lid.write(90)` tells the servo to
(a) spin 90 times (b) turn to 90 degrees (c) wait 90 seconds (d) stop
4. The test `d > 0` is included because
(a) 0 means very close (b) `pulseIn` returns 0 when no echo comes back (c) it looks neat (d) the servo needs it
5. The lid trembles while open. The most likely cause is
(a) wrong code (b) the lid is too heavy (c) the sensor is dirty (d) a missing LED
6. A machine acting when nobody intended it to is called a
(a) false trigger (b) short circuit (c) derived quantity (d) loop

B. Fill in the blanks

- 1 Never force a servo _____ round with your fingers.
- 2 The servo's ground must be _____ with the Arduino's ground.
- 3 In this project the lid opens when something is closer than _____ cm.
- 4 A part that moves or acts is called an _____.

C. Answer in one or two lines

- 1 Explain how a servo knows when it has reached the angle you asked for.
- 2 Name two things that could make your bin open when nobody is there.
- 3 Why is it worth watching a stranger use your machine?
- 4 Write down the `openDistance` you chose and why you chose it.

UNIT 5

Final Project: Obstacle-Avoiding Robot

Sessions 12, 13, 14 and 15

Everything you have built so far has stayed on the desk. This one drives away. A battery, two motors, a driver board and the sensor you already know combine into a machine that crosses a room on its own.

By the end of this unit you will be able to:

- Explain how an H-bridge reverses a motor.
- Separate logic power from motor power while sharing a ground.
- Write reusable motion functions instead of repeating code.
- Debug a system where several subsystems interfere with each other.

SESSION 12

one class

Motors, the H-Bridge and Battery Safety

Today you will

- Learn how four switches can reverse a motor.
- Read the L298N direction table.
- Agree the battery rules and the team roles.

To make a motor spin one way you send current through it in one direction. To reverse it you send the current the other way. An **H-bridge** is a clever arrangement of four electronic switches that lets a circuit do exactly that, on command, without anyone rewiring anything. It is called an H-bridge because in a circuit diagram the four switches and the motor form the shape of the letter H.

The L298N: two switches decide which way a motor turns

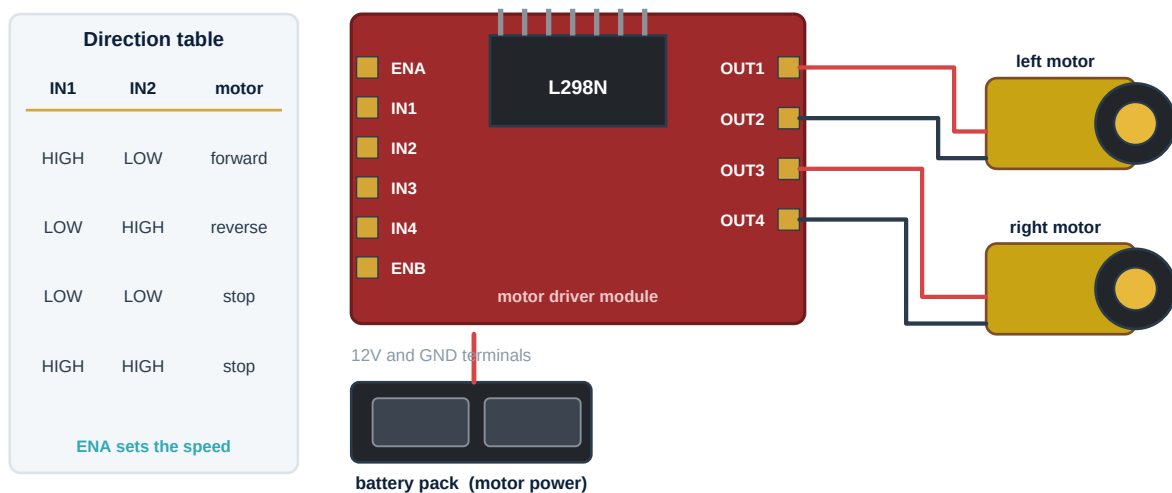


Figure 5.1 - IN pins choose the direction, EN pins choose the speed

The L298N contains two complete H-bridges, one per motor. Each motor needs two direction pins (IN1 and IN2 for the left, IN3 and IN4 for the right) and one enable pin (ENA, ENB) that controls speed. The direction table on the module tells you everything: one pin HIGH and the other LOW gives one direction, swap them for the other, and both the same means stop.

Watch Out

The L298N has small jumper caps over the ENA and ENB pins. While those caps are fitted, the motors run at full speed and your speed control does nothing at all. **Remove both caps** before wiring ENA and ENB to the Arduino.

12.1 Two Power Supplies, One Ground

This is the single most important idea in the whole unit. Your robot has **two** sources of power. The USB or the battery feeds the Arduino, which needs very little. The battery pack feeds the L298N, which needs a great deal. These are separate on purpose - motors produce electrical noise that would otherwise reset the Arduino mid-drive.

But there is a catch. For the Arduino's signals to mean anything to the L298N, both must agree on what zero volts is. So you must run one wire from **Arduino GND to L298N GND**. Without it, everything looks correctly wired and absolutely nothing works. More school robots fail on this one missing wire than on any other fault.

Role	Responsible for	Name
Chassis builder	Mounting motors, wheels, caster and battery firmly	_____

Role	Responsible for	Name
Wiring engineer	Every connection, checked twice, and the shared ground	_____
Programmer	Typing, uploading and tuning the turn timings	_____
Recorder	The notebook: test runs, faults, and what fixed them	_____

SESSION 13

one class

Building the Drive Base

Today you will

- Assemble the chassis, motors, wheels and battery.
- Wire the L298N and test all four directions.
- Write reusable motion functions.

Where everything sits on the robot

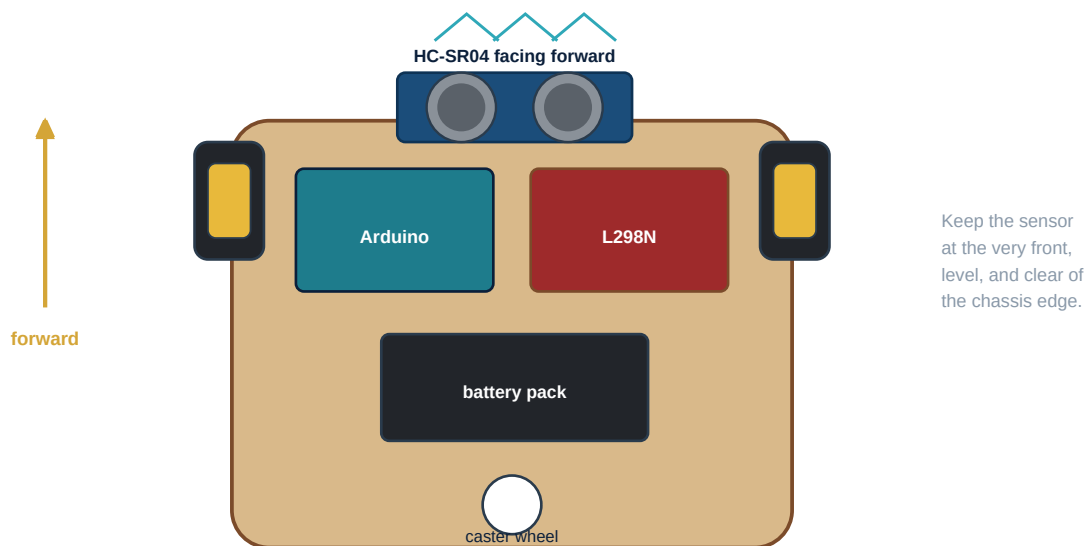


Figure 5.2 - Plan the layout before you cut a single cable tie

- Mount both motors so the wheels are **parallel**. A few degrees out and the robot curves instead of going straight.
- Put the battery low and central - a high, off-centre battery makes the robot tip when it turns.
- Leave the caster wheel free to swivel. If it jams, the robot fights itself.
- Route wires so nothing can be caught by a turning wheel. Use tape or cable ties.

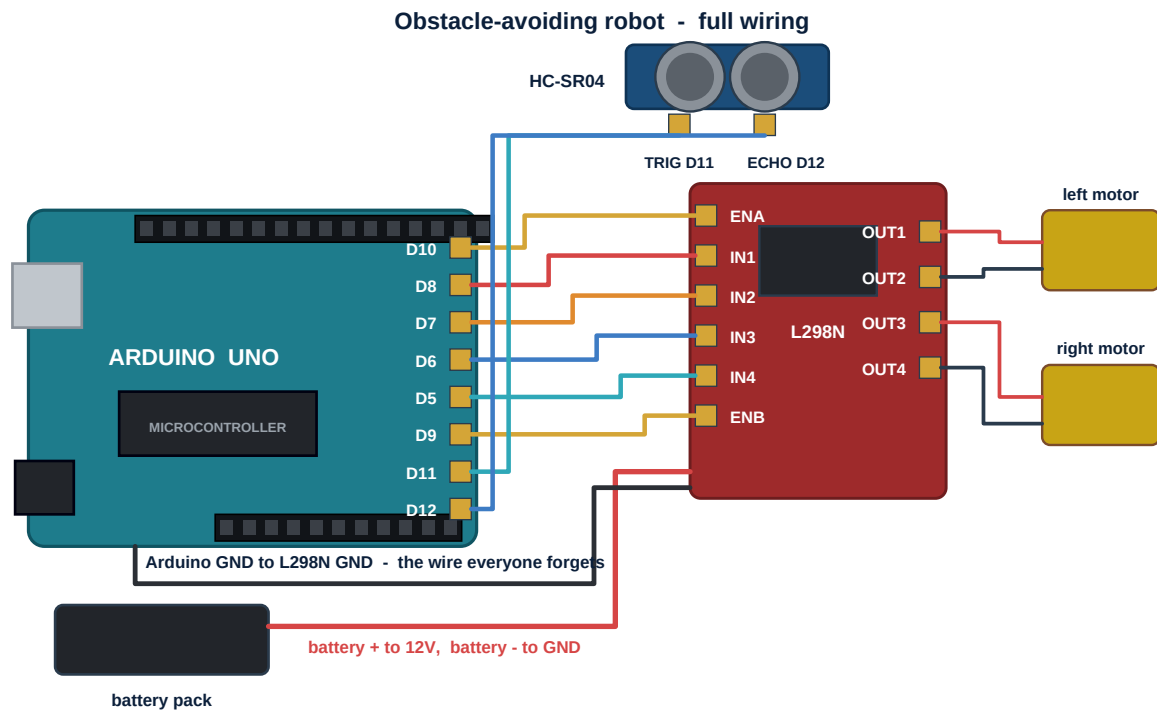


Figure 5.3 - Two power supplies, one shared ground

Connection	Goes to	Why
L298N IN1 / IN2	D8 / D7	Left motor direction
L298N IN3 / IN4	D6 / D5	Right motor direction
L298N ENA / ENB	D10 / D9	Speed for each motor
L298N OUT1 / OUT2	Left motor terminals	Power to the left wheel
L298N OUT3 / OUT4	Right motor terminals	Power to the right wheel
L298N 12V / GND	Battery + / Battery -	Motor power
L298N GND	Arduino GND	The shared ground. Never leave this out.
HC-SR04 TRIG / ECHO	D11 / D12	Obstacle detection

DriveBaseTest.ino - test the base before adding the sensor

```

int in1 = 8, in2 = 7, in3 = 6, in4 = 5;
int ena = 10, enb = 9;
int speed = 150;           // 0 to 255. Start slow.

void setup() {
  pinMode(in1, OUTPUT); pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT); pinMode(in4, OUTPUT);
  pinMode(ena, OUTPUT); pinMode(enb, OUTPUT);
}

void forward() {
  analogWrite(ena, speed); analogWrite(enb, speed);
  digitalWrite(in1, HIGH); digitalWrite(in2, LOW);
  digitalWrite(in3, HIGH); digitalWrite(in4, LOW);
}

void back() {
  analogWrite(ena, speed); analogWrite(enb, speed);
  digitalWrite(in1, LOW); digitalWrite(in2, HIGH);
  digitalWrite(in3, LOW); digitalWrite(in4, HIGH);
}

```

```
void turnRight() {           // left wheel forward, right wheel back
  analogWrite(ena, speed); analogWrite(enb, speed);
  digitalWrite(in1, HIGH); digitalWrite(in2, LOW);
  digitalWrite(in3, LOW);  digitalWrite(in4, HIGH);
}

void stopMotors() {
  analogWrite(ena, 0); analogWrite(enb, 0);
}

void loop() {                // a simple test of all four
  forward(); delay(1500);
  back();    delay(1500);
  turnRight(); delay(800);
  stopMotors(); delay(2000);
}
```

Watch Out

If a wheel spins the wrong way, do not rewrite the code. Simply swap that motor's two wires at the L298N terminals. It is faster and it is what a professional would do.

SESSION 14

one class

Adding the Eyes and the Algorithm

Today you will

- Mount the ultrasonic sensor at the front.
- Write the avoidance algorithm.
- Run the robot for the first time.

The robot's decision is simpler than people expect. Drive forward. Keep looking ahead. If something is closer than a set distance, stop, reverse a little to make room, turn, and carry on. Written as a flowchart before it is written as code, it takes five boxes.

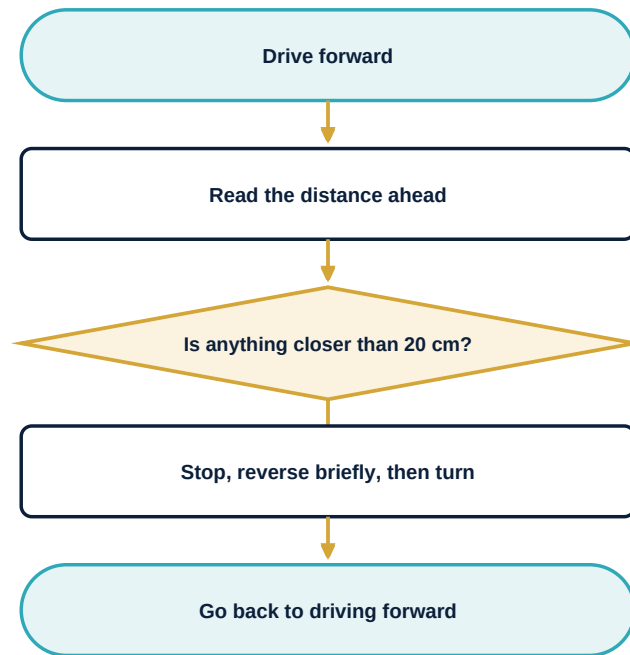


Figure 5.4 - The avoidance algorithm, in five steps

ObstacleAvoidingRobot.ino

```

int trig = 11, echo = 12;
int in1 = 8, in2 = 7, in3 = 6, in4 = 5;
int ena = 10, enb = 9;
int speed = 150;
int safeDistance = 20;          // centimetres

void setup() {
  pinMode(in1, OUTPUT); pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT); pinMode(in4, OUTPUT);
  pinMode(ena, OUTPUT); pinMode(enb, OUTPUT);
  pinMode(trig, OUTPUT); pinMode(echo, INPUT);
  Serial.begin(9600);
  delay(2000);                  // time to put the robot down and step back
}

int readDistance() {
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long t = pulseIn(echo, HIGH, 25000); // give up after 25 ms
  int cm = t * 0.034 / 2;
  if (cm == 0) { cm = 400; }           // no echo means nothing is near
  return cm;
}

void forward() {
  analogWrite(ena, speed); analogWrite(enb, speed);
  digitalWrite(in1, HIGH); digitalWrite(in2, LOW);
  digitalWrite(in3, HIGH); digitalWrite(in4, LOW);
}

void back() {
  analogWrite(ena, speed); analogWrite(enb, speed);

```

```

    digitalWrite(in1, LOW);    digitalWrite(in2, HIGH);
    digitalWrite(in3, LOW);    digitalWrite(in4, HIGH);
}

void turnRight() {
    analogWrite(ena, speed);    analogWrite(enb, speed);
    digitalWrite(in1, HIGH);    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);     digitalWrite(in4, HIGH);
}

void stopMotors() {
    analogWrite(ena, 0);    analogWrite(enb, 0);
}

void loop() {
    int d = readDistance();
    Serial.println(d);

    if (d < safeDistance) {
        stopMotors();    delay(200);
        back();          delay(400);    // make room to turn
        stopMotors();    delay(200);
        turnRight();     delay(500);    // tune this number on your own floor
        stopMotors();    delay(200);
    } else {
        forward();
    }

    delay(60);
}

```

Two details deserve attention. The third argument to `pulseIn(echo, HIGH, 25000)` is a **timeout**: if no echo arrives within 25 milliseconds, give up rather than freezing the robot mid-drive. And the line turning a 0 into 400 says, in effect, *hearing nothing means nothing is there* - which is true, and which stops the robot spinning on the spot in an open room.

SESSION 15

one class

Arena Testing and Your Demonstration

Today you will

- Tune the turn timing on the actual floor.
- Pass every acceptance test.
- Present your robot in three minutes.

Your robot will behave differently on smooth tiles and on rough concrete, and differently again as the battery drains. Tuning is not cheating - it is the job. Work through the fault table before you start changing numbers at random.

What you see	Most likely cause	What to do
Nothing happens at all	Missing Arduino-to-L298N ground wire	Add it. This is the number one fault.
Motors run flat out, speed ignored	The ENA and ENB jumper caps are still fitted	Remove both caps
Robot curves instead of driving straight	One motor is faster than the other	Use a slightly lower speed value on the faster side

What you see	Most likely cause	What to do
Robot spins forever in an empty room	A 0 reading treated as very close	Keep the line that converts 0 into 400
Robot resets whenever motors start	Motor noise, or a weak battery	Charge or replace the cells; check the shared ground
Turns too little or too much	Turn delay not tuned for this floor	Adjust the delay after turnRight in steps of 100

Acceptance test	Result	Tick
Drives straight for 2 metres with reasonable drift	_____	■
Stops before touching a wall, every time	_____	■
Turns and continues without human help	_____	■
Crosses a 3 x 3 m arena with 5 obstacles, no rescue	_____	■
Runs for 3 minutes without resetting	_____	■

15.1 Your Three-Minute Demonstration

- The problem** - one sentence on what your machine does and why that is useful.
- How it senses** - hold up the HC-SR04 and explain the echo in your own words.
- How it decides** - show the line containing safeDistance and say what the number means.
- How it acts** - explain the H-bridge in one sentence, then run the robot live.
- What went wrong** - name one fault you found and fixed. Judges value this most.
- What next** - one honest improvement you would make with more time.

Challenge

Mount the HC-SR04 on the SG90 servo. When the robot meets an obstacle, stop, sweep the sensor left and right, compare the two distances, and turn towards whichever side has more space. This is how real robots choose a path rather than always turning the same way.

New word	What it means
H-bridge	Four switches that let a circuit reverse a motor on command.
Motor driver	A board that switches heavy motor current on a small signal's command.
Shared ground	The wire that lets two power supplies agree on what zero volts means.
Timeout	Giving up on waiting so a program is never stuck.
Algorithm	A clear, ordered set of steps a machine follows to solve a problem.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 5**A. Choose the correct answer**

1. An H-bridge lets a circuit
(a) measure distance (b) reverse a motor (c) store power (d) read a sensor
2. On the L298N, ENA and ENB control
(a) direction (b) speed (c) the sensor (d) the battery
3. The wire most often forgotten is
(a) TRIG to D11 (b) Arduino GND to L298N GND (c) the LED wire (d) the USB cable
4. If the motors ignore your speed setting, you should
(a) change the code (b) remove the ENA and ENB jumper caps (c) use a bigger battery (d) swap the motors
5. `pulseIn(echo, HIGH, 25000)` will give up after
(a) 25 microseconds (b) 25 milliseconds (c) 25 seconds (d) never
6. A reading of 0 from the sensor is converted to 400 because
(a) 400 is the maximum (b) no echo means nothing is nearby (c) it makes the robot faster (d) the code needs it

B. Fill in the blanks

- 1 Two pins set a motor's _____ and one pin sets its _____.
- 2 IN1 HIGH and IN2 LOW makes the motor turn _____.
- 3 A wheel spinning the wrong way is fixed by swapping its two _____.
- 4 An ordered set of steps a machine follows is called an _____.

C. Answer in one or two lines

- 1 Explain in your own words why the Arduino and the L298N must share a ground.
- 2 Your robot curves to the left while driving forward. Give two possible reasons.
- 3 Write down the turn delay you settled on and how you arrived at it.
- 4 Which of the three projects this year taught you the most, and why?

Extra Challenges

Finished early? Pick any of these. They all use parts you already have.

#	Challenge	Uses
1	A reversing alarm that beeps faster as an object comes closer	HC-SR04 + buzzer
2	A height station that reports feet and inches as well as centimetres	maths
3	A digital tape measure that holds the reading when you press a button	switch + LEDs
4	A servo that follows your hand: closer hand, larger angle	HC-SR04 + servo
5	A bin that also reports how full it is	second HC-SR04
6	A door that opens for people but stays shut for walls	servo + threshold
7	A robot that drives a square path with no sensor at all	L298N + timing
8	A robot that chooses left or right by sweeping a servo-mounted sensor	servo + HC-SR04
9	A speed test: measure how long the robot takes to cross 2 metres	millis()
10	A robot that beeps and flashes before every turn, like a real vehicle	buzzer + LEDs

MIXED REVISION

Twenty Questions to Test Yourself

- 1 What are the three steps every robot follows?
- 2 Why must a circuit be a complete loop?
- 3 How many holes are joined in one breadboard column?
- 4 How far does sound travel in one microsecond?
- 5 Write the four steps of the ultrasonic trigger sequence.
- 6 What does `pulseIn()` actually measure?
- 7 Why does the distance formula divide by two?
- 8 An echo returns in 1500 microseconds. How far away is the object?
- 9 Why should a timing value be stored in a long rather than an int?
- 10 What is a derived quantity? Give one example from this book.
- 11 What is noise, and how does averaging reduce it?
- 12 How does a servo know it has reached the angle you asked for?
- 13 Why must you never force a servo horn by hand?
- 14 Give two causes of a false trigger on the smart dustbin.
- 15 Why can a motor not be connected directly to an Arduino pin?
- 16 What does an H-bridge do?
- 17 Which L298N pins set direction, and which set speed?
- 18 Why must the Arduino and the L298N share a ground?
- 19 Why is a 0 reading from the sensor converted to 400 in the robot code?
- 20 Name one improvement you would make to your robot with one more session.

Words to Know

Word	Meaning
Actuator	Any part that moves or acts: a motor, a buzzer, a lamp.
Algorithm	A clear, ordered set of steps a machine follows to solve a problem.
Analog	A value that can be anything within a range, not just on or off.
Averaging	Taking several readings and using their mean to reduce noise.
Breadboard	A block of holes for joining parts without soldering.
Current	How much electricity flows each second. Measured in amps.
Debugging	Hunting down the reason something does not work.
Derived quantity	A useful number worked out from something else that was measured.
Digital	A signal with only two values: HIGH or LOW.
Error	The difference between a measured value and the true value.
False trigger	The machine acting when nobody intended it to.
Function	An instruction you write yourself and then reuse.
Ground (GND)	The return path. Every circuit needs one.
H-bridge	Four switches that let a circuit reverse a motor on command.
Horn	The plastic arm attached to a servo's shaft.
long	A variable type that holds very large whole numbers.
Microsecond	One millionth of a second.
Motor driver	A board that switches heavy motor current on a small signal's command.
Noise	Small random variation in a sensor's readings.
pulseIn()	Measures how long a pin stays HIGH, in microseconds.
PWM	Fast switching that makes an output appear to be in between on and off.
return	Hands a value back from a function to whoever called it.
Servo	A motor that turns to an exact angle and holds it.
Shared ground	The wire that lets two power supplies agree on what zero volts means.
Threshold	The number at which a decision changes from one answer to the other.
Time of flight	Measuring distance by timing how long a signal takes to return.
Timeout	Giving up on waiting so a program is never stuck.
Ultrasonic	Sound too high for human ears to hear.

Answer Key

Section A only. Sections B and C are for you and your trainer to discuss - there is often more than one good answer.

Check Yourself - Unit 1

Q	Answer	The correct option
1	(c)	a complete loop
2	(c)	the plus side
3	(a)	columns of five
4	(b)	analog
5	(a)	20 milliamps
6	(b)	error

Check Yourself - Unit 2

Q	Answer	The correct option
1	(b)	a millionth of a second
2	(b)	how long the pin stays HIGH
3	(c)	long
4	(b)	return
5	(b)	it draws far too much current
6	(b)	5 times

Check Yourself - Unit 3

Q	Answer	The correct option
1	(b)	timing an echo
2	(b)	sound travels there and back
3	(b)	10 microseconds
4	(b)	TRIG and ECHO swapped
5	(b)	138 cm
6	(b)	noise

Check Yourself - Unit 4

Q	Answer	The correct option
1	(b)	knows its own angle
2	(c)	orange
3	(b)	turn to 90 degrees
4	(b)	pulseIn returns 0 when no echo comes back
5	(b)	the lid is too heavy
6	(a)	false trigger

Check Yourself - Unit 5

Q	Answer	The correct option
1	(b)	reverse a motor
2	(b)	speed
3	(b)	Arduino GND to L298N GND
4	(b)	remove the ENA and ENB jumper caps
5	(b)	25 milliseconds
6	(b)	no echo means nothing is nearby

My Project Log

Fill one row after every class. This is your evidence, and it counts towards your assessment.

Session	What I built or learnt	A problem I hit	How I fixed it
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			

A last word

This year your machines learnt to measure honestly and then to move. Notice how much of the work was not writing code at all: mounting a sensor straight, keeping a lid light, remembering one ground wire. That is real engineering. The code is usually the easy part.